

University of Moratuwa

Faculty of Engineering



EN3150 – Pattern Recognition

Assignment 03

Resource-Constrained CNN for Edge Image Classification

Group Name - Neural Nexus

Name	Index Number
Dilhara D.S.	230147L
DISSANAYAKE G.R.G.K.	230159B
MENDIS T.A.S.D.	230409T
MARASINGHA M.D.S.	230398F

Contents

1	Dataset Overview: MNIST 2-Digit Dataset	1
1.1	Overview	1
1.2	Characteristics of the Dataset	1
1.3	Image Pre-processing	2
1.4	Train-Validation-Test Split	2
1.5	Sample Images	3
1.6	Dataset Summary	3
2	Custom CNN for Handwritten Digit Classification	4
2.1	Dataset Loading	4
2.2	Dataset Pre-Processing	4
2.3	Train-Validation-Test Split	4
2.4	CNN Model Architecture	4
2.4.1	Model A Architecture	4
2.4.2	Model A Architecture Summary	6
2.5	Justification for Activation Functions	6
2.6	Why Adam Optimizer is Used?	6
2.7	Learning Rate Selection	6
2.8	CNN Model Training	7
2.9	Model A Training Results	7
2.10	Model B Learning Curves	8
2.11	Optimizer Comparison	9
2.11.1	Comparison Metrics	9
2.11.2	Optimizer Results	9
2.12	Impact of Momentum Parameter	10
2.12.1	Comparison of Momentum Behaviour	10
2.12.2	Key Observations	10
2.13	Model Evaluation on Test Dataset	10
2.13.1	Overall Performance Metrics	11
2.13.2	Confusion Matrix Analysis	11
2.13.3	Analysis and Insights	12

3	Resource-Constrained CNN	13
3.1	Motivation for a Lightweight Model	13
3.2	Depthwise-Separable Convolution	13
3.3	Model B Architecture	13
3.4	Parameter Constraint Verification	14
3.5	Model B Learning Curves	15
3.6	Model B Evaluation	16
3.7	Model B Training Behaviour	16
3.8	Model B Confusion Matrix	17
3.9	Model B Analysis	17
4	Comparison with State-of-the-Art Networks (Transfer Learning)	18
4.1	Selection and Justification of Pre-trained Models	18
4.1.1	Model 1: EfficientNet-B0	18
4.1.2	Model 2: MobileNetV2	18
4.2	Fine-Tuning Strategy and Implementation	18
4.2.1	Data Preprocessing Pipeline	18
4.2.2	Model Adaptation Methodology	19
4.2.3	Transfer Learning Approach	19
4.3	Individual Model Performance Evaluation	19
4.3.1	MobileNetV2 Fine-tuned Model Evaluation	19
4.3.2	EfficientNet-B0 Fine-tuned Model Evaluation	20
4.4	Performance Comparison: Custom CNN vs. Fine-tuned State-of-the-Art Models	22
4.5	Point-by-Point Comparative Analysis	22
4.5.1	Accuracy	22
4.5.2	Memory Footprint	23
4.5.3	Computational Cost	24
4.6	Key Findings and Interpretations	24
4.7	Strategic Implications	24
4.8	Conclusion	25
4.9	Trade-offs, Advantages, and Limitations: Custom Model vs. Pre-trained Model	25
4.9.1	Custom CNN Model Analysis	25
4.9.2	Pre-trained Model Analysis	26
4.9.3	Decision Framework: When to Choose Which Approach	26
4.9.4	Strategic Trade-offs Summary	26
4.9.5	Conclusion: Balanced Strategy	27

5 Overall Conclusion	28
A Appendix	30
A.1 GitHub URL	30
A.2 Final Model Comparison	30

1 Dataset Overview: MNIST 2-Digit Dataset

1.1 Overview

The dataset used in this project is the MNIST 2-Digit Dataset. The dataset contains handwritten images representing two-digit numbers.

Unlike the original MNIST dataset, which contains ten individual digit classes, the two-digit dataset represents numbers using two digits. Therefore, the classification task contains 100 possible classes, ranging from 00 to 99.

The objective of this assignment is to investigate convolutional neural network architectures for two-digit handwritten number classification while considering memory and computational constraints.

The project evaluates both custom CNN architectures and fine-tuned pre-trained models.

1.2 Characteristics of the Dataset

The dataset provides image files together with CSV metadata.

The metadata contains two digit labels:

- `label_1`: First digit.
- `label_2`: Second digit.

The final class label is calculated as:

$$\text{Class Label} = 10 \times \text{label_1} + \text{label_2}$$

Therefore:

$$00, 01, 02, \dots, 98, 99$$

represent the 100 possible classes.

1.3 Image Pre-processing

The assignment specifies a maximum image resolution of 64×64 pixels.

Therefore, all images were resized to:

$$64 \times 64$$

The preprocessing pipeline consisted of:

1. Loading image metadata.
2. Constructing the two-digit class label.
3. Locating the corresponding image files.
4. Resizing images to 64×64 pixels.
5. Converting images into tensors.
6. Normalizing the image data.

1.4 Train-Validation-Test Split

The dataset was divided into three subsets:

- 70% training data
- 15% validation data
- 15% test data

A stratified split was used so that the class distribution was maintained as consistently as possible across the training, validation, and testing subsets.

1.5 Sample Images

Figure 1.1 shows examples of handwritten two-digit images from the dataset.

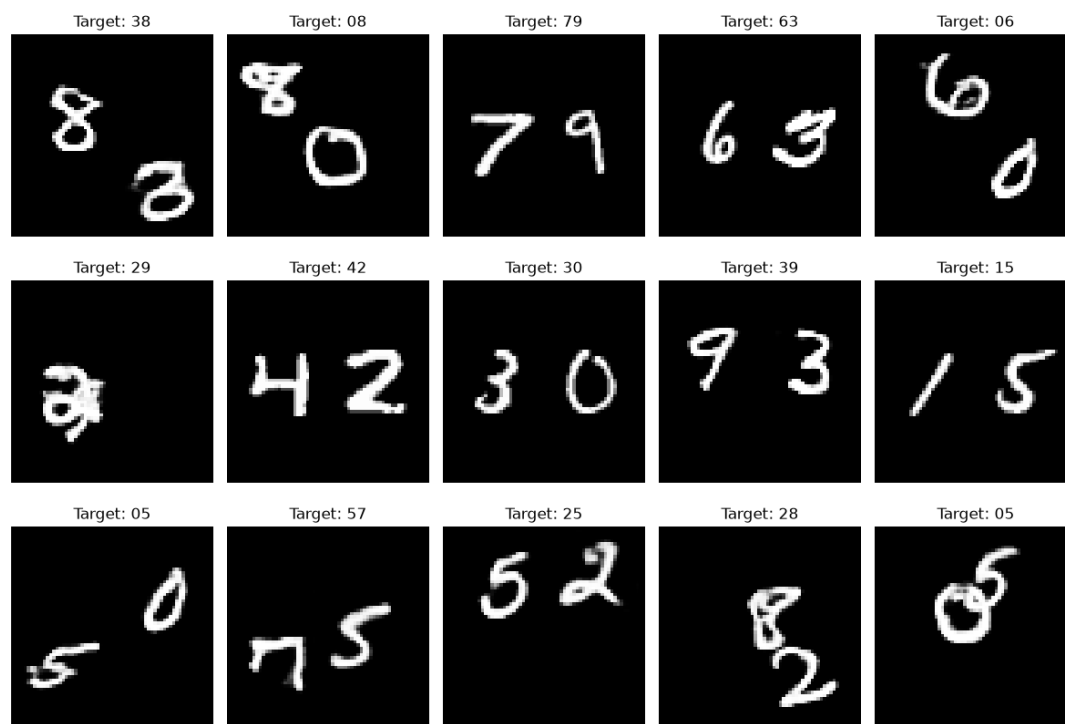


Figure 1.1: Sample images from the MNIST 2-Digit Dataset.

1.6 Dataset Summary

Table 1.1: Summary of the MNIST 2-Digit Dataset

Property	Value
Dataset	MNIST 2-Digit Dataset
Task	Two-digit number classification
Number of Classes	100
Class Range	00–99
Maximum Image Resolution	64×64
Training Split	70%
Validation Split	15%
Testing Split	15%

2 Custom CNN for Handwritten Digit Classification

2.1 Dataset Loading

The dataset metadata was loaded from the provided CSV files.

The CSV files contain the image filename and the two individual digit labels.

The image filename was matched recursively with the corresponding image file.

The two labels were combined into one 100-class target label.

2.2 Dataset Pre-Processing

Each image was converted into a grayscale image and resized to 64×64 pixels.

The preprocessing pipeline was kept consistent between the custom CNN models.

This ensures that differences in performance are primarily caused by architectural differences rather than different input processing procedures.

2.3 Train-Validation-Test Split

The same 70/15/15 split was used for all custom models.

This is important because the models must be evaluated using the same test distribution to provide a fair comparison.

2.4 CNN Model Architecture

Two custom CNN architectures were developed.

- Model A: Standard CNN.
- Model B: Lightweight CNN using depthwise-separable convolutions.

2.4.1 Model A Architecture

Model A is a standard convolutional neural network consisting of three convolutional blocks followed by fully connected layers.

The architecture is:

1. Conv2D: $1 \rightarrow 16$
2. ReLU
3. MaxPooling
4. Conv2D: $16 \rightarrow 32$
5. ReLU
6. MaxPooling
7. Conv2D: $32 \rightarrow 64$
8. ReLU
9. MaxPooling
10. Fully Connected: $64 \times 8 \times 8 \rightarrow 128$
11. ReLU
12. Fully Connected: $128 \rightarrow 100$

The final layer produces 100 class scores.

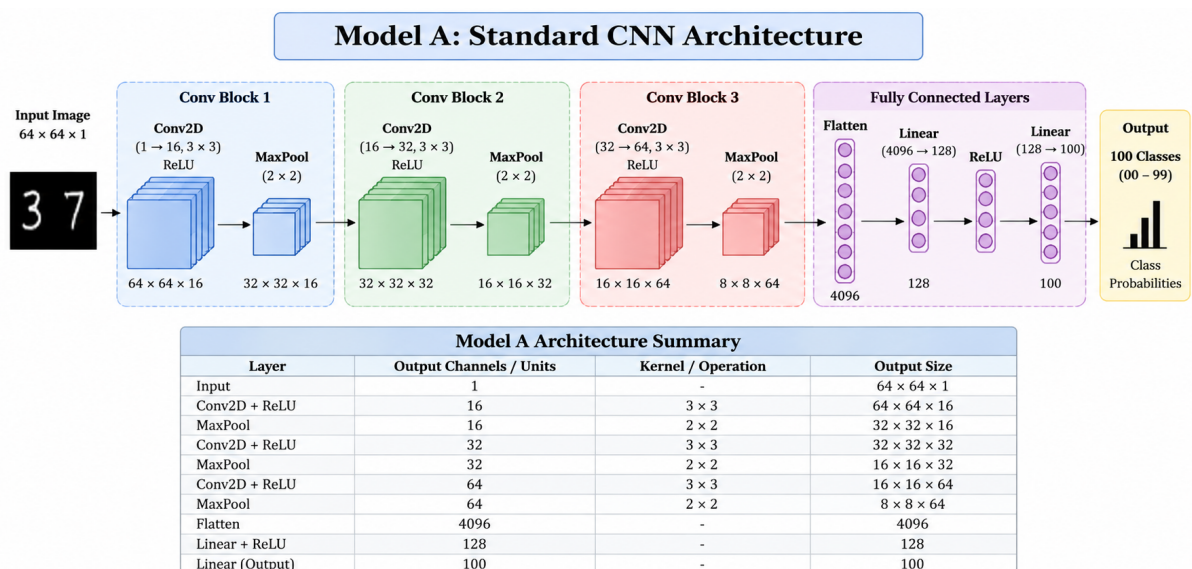


Figure 2.1: Model A Architecture

2.4.2 Model A Architecture Summary

Table 2.1: Model A Architecture

Layer	Output Channels	Operation
Input	1	Grayscale image
Conv1	16	3×3 convolution
Pool1	16	Max pooling
Conv2	32	3×3 convolution
Pool2	32	Max pooling
Conv3	64	3×3 convolution
Pool3	64	Max pooling
FC1	128	Fully connected
Output	100	Classification

2.5 Justification for Activation Functions

The ReLU activation function was used throughout the custom CNN.

The ReLU function is:

$$f(x) = \max(0, x)$$

ReLU is computationally simple and introduces non-linearity into the network.

It also avoids some of the optimization difficulties associated with saturating activation functions.

2.6 Why Adam Optimizer is Used?

Adam was selected as the primary optimizer for the main CNN experiments.

Adam combines adaptive learning rates with momentum-based optimization.

This makes it suitable for training CNNs efficiently without requiring extensive manual learning-rate adjustment.

2.7 Learning Rate Selection

The learning rate controls the size of the parameter updates during training.

A learning rate that is too large may result in unstable optimization, while a very small learning rate may result in slow convergence.

The selected learning rate was used consistently for the main model comparison.

2.8 CNN Model Training

The custom CNN models were trained for 20 epochs.

Cross-entropy loss was used for the 100-class classification problem.

The training process monitored:

- Training loss
- Training accuracy
- Validation loss
- Validation accuracy

2.9 Model A Training Results

Model A achieved strong performance during training.

The final test results are shown in Table 2.2.

Table 2.2: Model A Test Performance

Metric	Result
Trainable Parameters	560,612
Model Size	2.1431 MB
Average Time/Epoch	77.48 s
Test Accuracy	94.06%
Macro Precision	93.91%
Macro Recall	94.94%

2.10 Model B Learning Curves

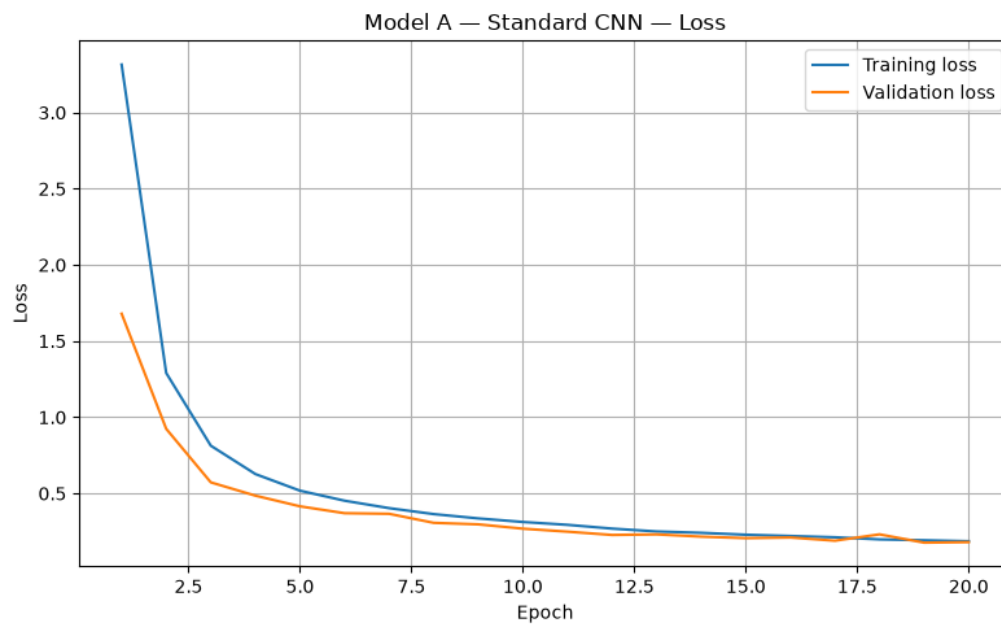


Figure 2.2: Losses of Training and validation curves for Model A.

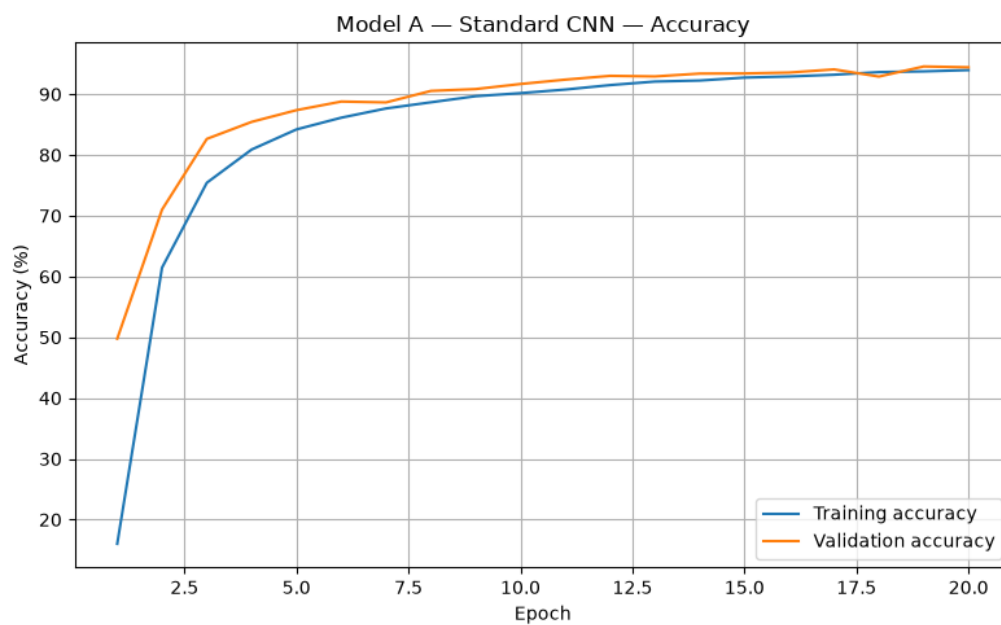


Figure 2.3: Accuracy Training and validation curves for Model A.

2.11 Optimizer Comparison

Three optimizers were investigated:

1. Adam
2. SGD
3. SGD with Momentum

A five-epoch comparison was conducted to study their early optimization behaviour.

2.11.1 Comparison Metrics

The main metrics used for the optimizer comparison were:

- Training accuracy
- Validation accuracy
- Training loss
- Validation loss

2.11.2 Optimizer Results

The optimizer comparison produced the following results.

Table 2.3: Five-Epoch Optimizer Comparison

Optimizer	Best Validation Accuracy	Final Validation Accuracy
SGD	4.89%	4.90%
SGD + Momentum	14.86%	5.62%
Adam	10.15%	7.92%

The SGD with Momentum configuration reached a peak validation accuracy of approximately 14.87% during the short experiment. However, its final validation accuracy dropped to approximately 5.62% by epoch 5.

Adam reached 48.17% training accuracy by epoch 5, while SGD and SGD with Momentum reached 4.77% and 25.09%, respectively.

2.12 Impact of Momentum Parameter

Momentum influences the direction and magnitude of parameter updates.

The general momentum update can be represented as:

$$v_t = \beta v_{t-1} + (1 - \beta)g_t$$

where:

- v_t is the current velocity,
- β is the momentum coefficient,
- g_t is the current gradient.

Momentum can improve convergence, but inappropriate settings can also cause unstable optimization.

2.12.1 Comparison of Momentum Behaviour

The results indicate that momentum can produce rapid improvements during the early training stage.

However, the validation accuracy did not remain stable throughout the short five-epoch experiment.

Therefore, a longer experiment would be required before making a final conclusion about the best momentum value.

2.12.2 Key Observations

The main observations are:

- Optimizer choice strongly affects convergence.
- Momentum can improve early learning.
- Validation performance can fluctuate significantly.
- Five epochs are insufficient for a definitive optimizer benchmark.

2.13 Model Evaluation on Test Dataset

After training, the final models were evaluated using the independent test dataset.

The following metrics were calculated:

- Accuracy
- Macro precision
- Macro recall
- Confusion matrix

2.13.1 Overall Performance Metrics

Model A achieved:

$$Accuracy = 94.06\%$$

$$Precision = 93.91\%$$

$$Recall = 94.94\%$$

These results indicate strong classification performance across the 100 classes.

2.13.2 Confusion Matrix Analysis

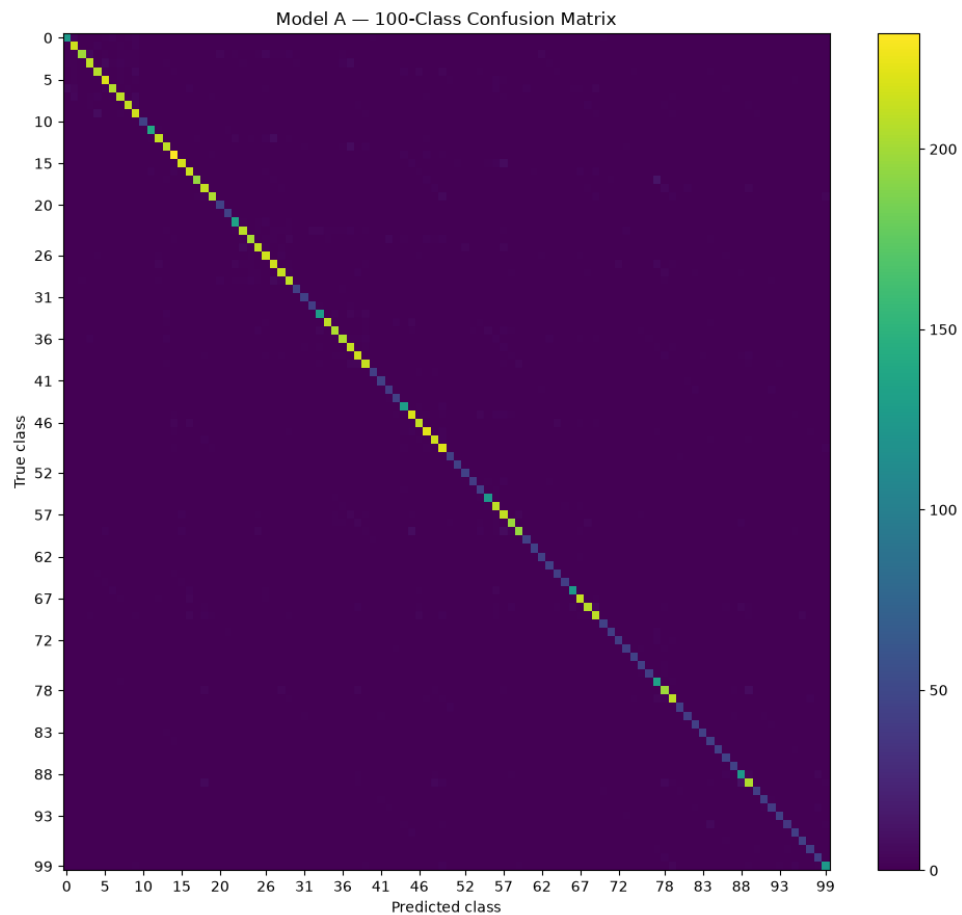


Figure 2.4: Confusion matrix of Model A on the test dataset.

2.13.3 Analysis and Insights

The high accuracy, precision, and recall indicate that Model A successfully learned useful visual features from the handwritten two-digit images.

The model provides a good balance between model complexity and classification performance.

3 Resource-Constrained CNN

3.1 Motivation for a Lightweight Model

Edge devices often have limited memory, processing power, and energy availability.

Therefore, a model with a large number of parameters may not be appropriate for deployment on resource-constrained hardware.

The assignment requires a custom CNN with fewer than 100,000 trainable parameters.

Model B was therefore designed specifically to satisfy this constraint.

3.2 Depthwise-Separable Convolution

Model B uses depthwise-separable convolutions.

A depthwise-separable convolution divides a conventional convolution into:

1. Depthwise convolution.
2. Pointwise convolution.

This reduces the number of parameters and computational operations compared with standard convolution.

3.3 Model B Architecture

The architecture consists of:

1. Depthwise-separable convolution: $1 \rightarrow 16$
2. Max pooling
3. Depthwise-separable convolution: $16 \rightarrow 32$
4. Max pooling
5. Depthwise-separable convolution: $32 \rightarrow 64$
6. Max pooling
7. Adaptive average pooling
8. Fully connected layer: $64 \rightarrow 100$

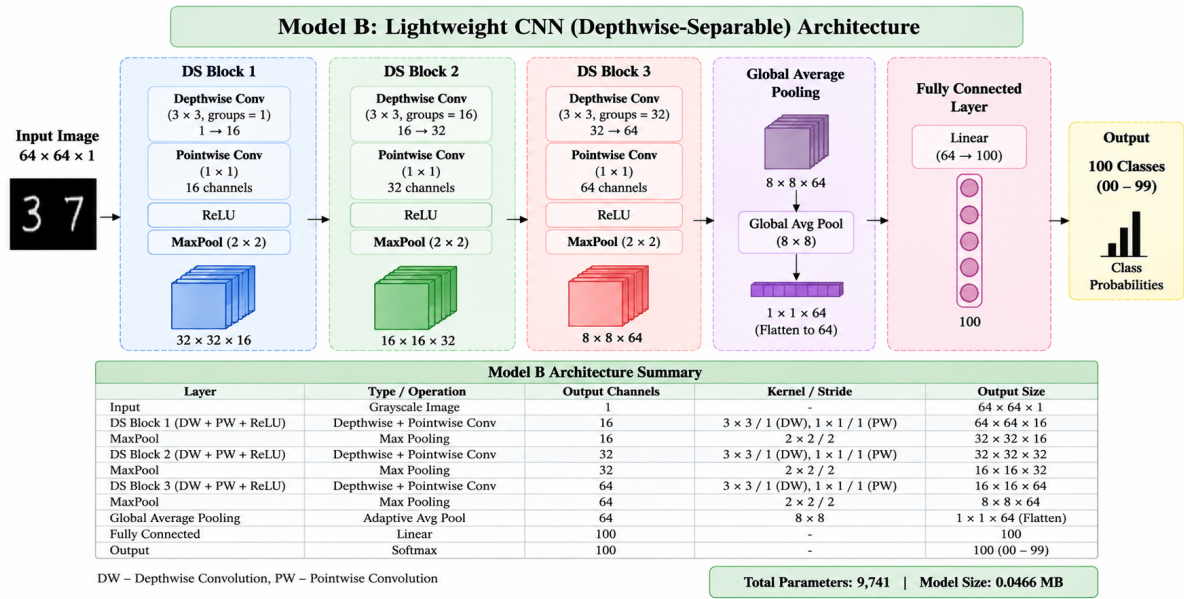


Figure 3.1: Lightweight Model

3.4 Parameter Constraint Verification

The assignment requires:

$$\text{Trainable Parameters} < 100,000$$

Model B contains:

$$9,741$$

trainable parameters.

Therefore:

$$9,741 < 100,000$$

The model successfully satisfies the required resource constraint.

3.5 Model B Learning Curves

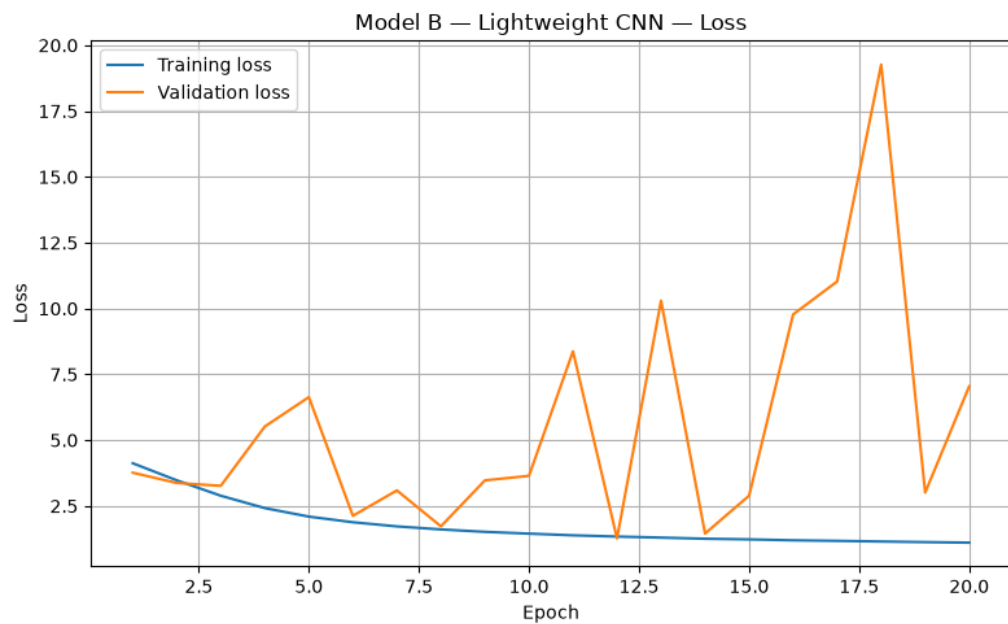


Figure 3.2: Losses of Training and validation curves for Model B.

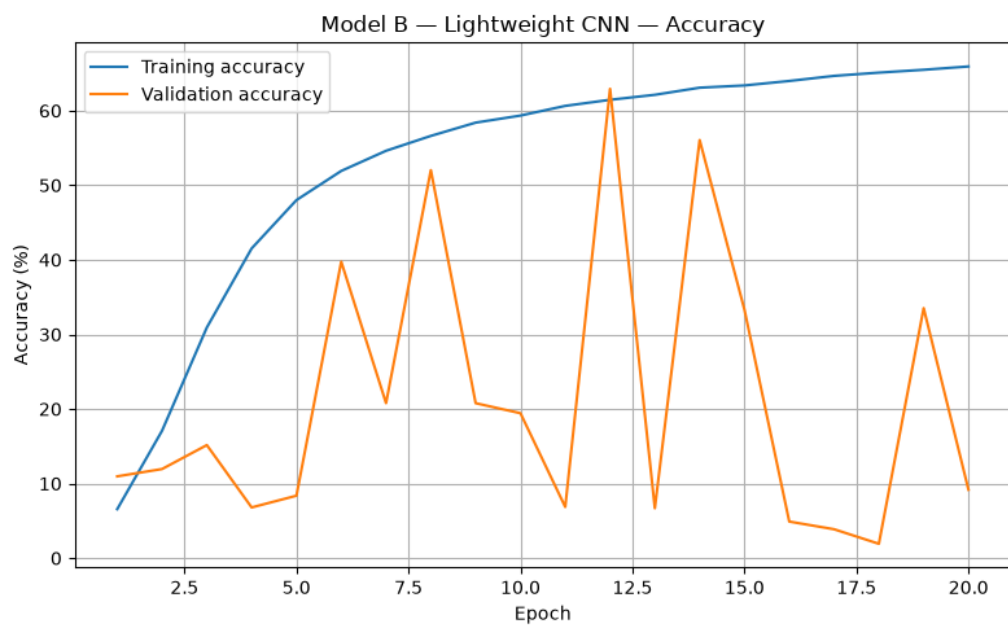


Figure 3.3: Accuracy Training and validation curves for Model B.

3.6 Model B Evaluation

The final Model B results are shown below.

Table 3.1: Model B Test Performance

Metric	Result
Trainable Parameters	9,741
Model Size	0.0466 MB
Average Time/Epoch	69.69 s
Test Accuracy	62.64%
Macro Precision	51.69%
Macro Recall	47.87%

3.7 Model B Training Behaviour

The training accuracy increased to approximately 65.93% by epoch 20.

However, the final validation accuracy decreased to approximately 9.18% at epoch 20, showing substantial instability in validation performance during the lightweight-model training.

This indicates instability near the end of training and suggests that further regularization or learning-rate tuning may improve generalization.

3.8 Model B Confusion Matrix

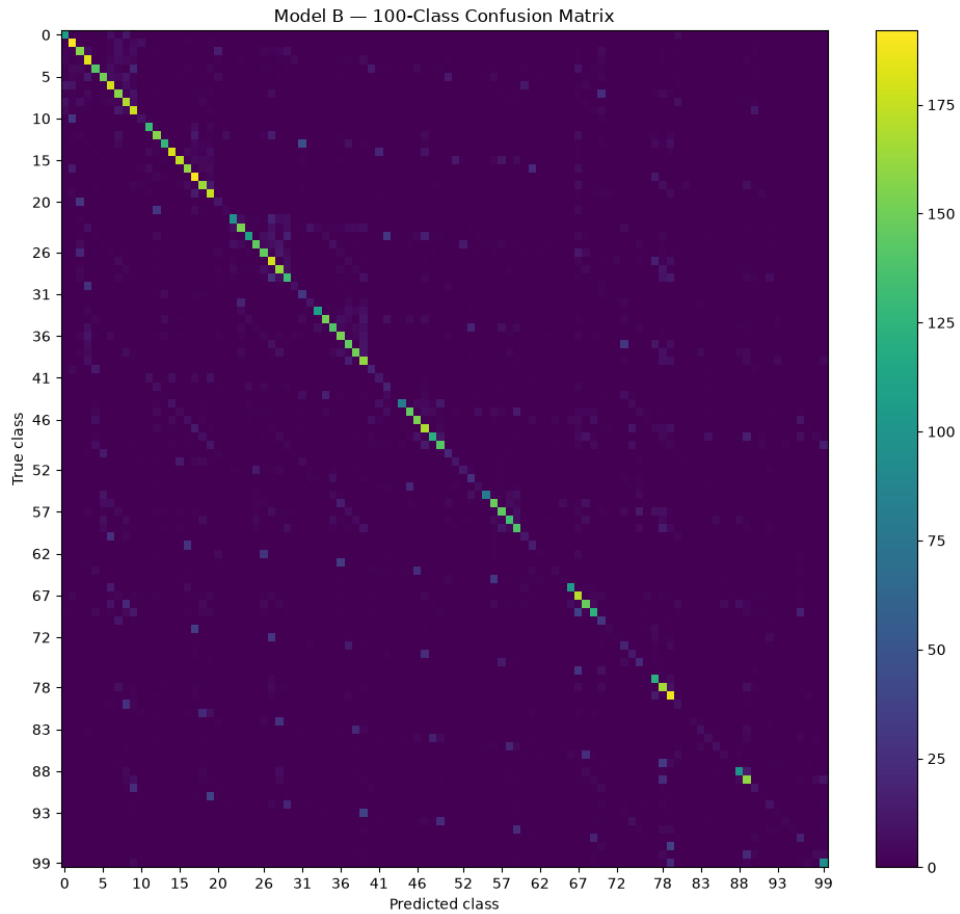


Figure 3.4: Confusion matrix of Model B on the test dataset.

3.9 Model B Analysis

Model B provides a very small memory footprint.

It contains only 9,741 trainable parameters compared with 560,612 parameters in Model A.

Therefore, Model B has approximately 57.6 times fewer trainable parameters than Model A.

However, this large reduction in model complexity resulted in a substantial reduction in classification accuracy.

Model B achieved 62.64% accuracy compared with 94.06% for Model A.

This demonstrates the trade-off between resource usage and classification performance.

4 Comparison with State-of-the-Art Networks (Transfer Learning)

4.1 Selection and Justification of Pre-trained Models

Two established convolutional neural network architectures were selected for transfer learning:

1. EfficientNet-B0
2. MobileNetV2

These architectures were selected because they are widely used for efficient image classification and provide a useful comparison with the custom resource-constrained architecture.

4.1.1 Model 1: EfficientNet-B0

EfficientNet-B0 uses compound scaling to balance network depth, width, and input resolution.

It provides strong classification performance while maintaining a relatively efficient architecture compared with many larger CNNs.

4.1.2 Model 2: MobileNetV2

MobileNetV2 was designed specifically for efficient computation on mobile and embedded devices.

It uses inverted residual blocks and linear bottlenecks to reduce computational cost.

4.2 Fine-Tuning Strategy and Implementation

Both pre-trained models were adapted for the 100-class two-digit classification problem. The final classification layer was replaced with a new layer producing 100 output classes.

4.2.1 Data Preprocessing Pipeline

The custom dataset images are grayscale.

The pre-trained networks expect three-channel input.

Therefore, the grayscale images were converted into a three-channel representation before being passed to the pre-trained networks.

All models used the same underlying train, validation, and test data split.

4.2.2 Model Adaptation Methodology

The classifier layer of each pre-trained network was replaced with a 100-class output layer.

The models were then fine-tuned using the project dataset.

The objective was to determine whether transfer learning could provide higher classification accuracy than the custom CNNs.

4.2.3 Transfer Learning Approach

The pre-trained ImageNet weights provide an initial set of learned visual features.

Fine-tuning then adapts these representations to the two-digit handwritten classification problem.

This approach provides a useful comparison between:

- Learning features completely from scratch.
- Reusing features learned from a large external dataset.

4.3 Individual Model Performance Evaluation

The two pre-trained models were trained for 20 epochs and evaluated on the same test set.

4.3.1 MobileNetV2 Fine-tuned Model Evaluation

The final MobileNetV2 results are:

Table 4.1: MobileNetV2 Performance

Metric	Result
Trainable Parameters	2,351,972
Model Size	9.2100 MB
Average Time/Epoch	101.95 s
Test Accuracy	97.14%
Macro Precision	97.04%
Macro Recall	97.56%

MobileNetV2 achieved higher accuracy than both custom CNN architectures.

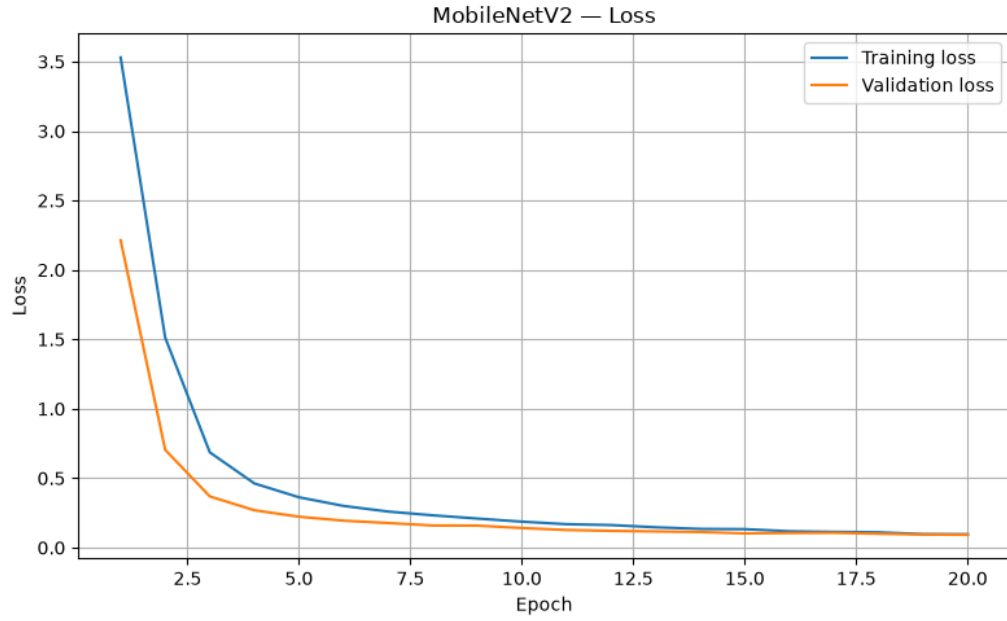


Figure 4.1: Losses of Training and validation curves for MobileNetV2.

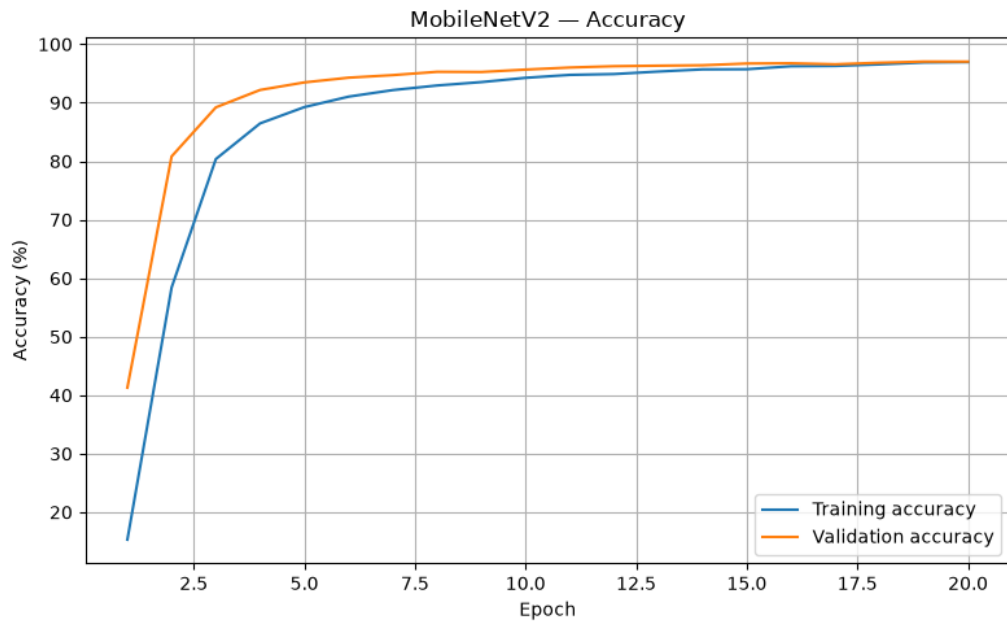


Figure 4.2: Accuracy of Training and validation curves for MobileNetV2.

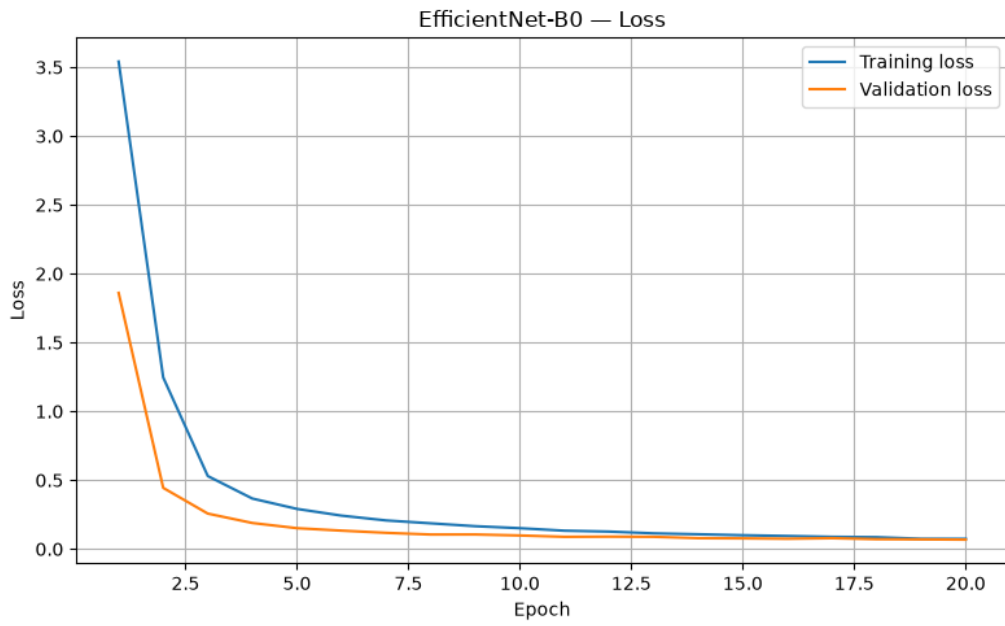
4.3.2 EfficientNet-B0 Fine-tuned Model Evaluation

The final EfficientNet-B0 results are:

Table 4.2: EfficientNet-B0 Performance

Metric	Result
Trainable Parameters	4,135,648
Model Size	16.0674 MB
Average Time/Epoch	118.78 s
Test Accuracy	97.82%
Macro Precision	97.60%
Macro Recall	98.10%

EfficientNet-B0 achieved the highest test accuracy among the four evaluated models.

**Figure 4.3:** Losses of Training and validation curves for EfficientNet-B0.

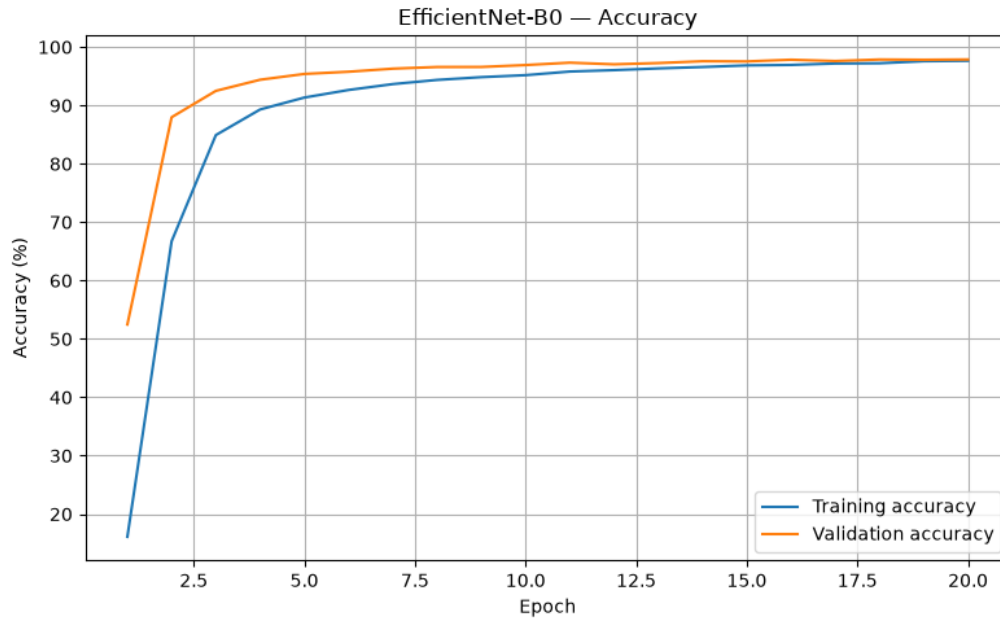


Figure 4.4: Accuracy of Training and validation curves for EfficientNet-B0.

4.4 Performance Comparison: Custom CNN vs. Fine-tuned State-of-the-Art Models

The complete comparison is shown in Table 4.3.

Table 4.3: Comprehensive Performance Comparison

Model	Parameters	Size (MB)	Time/Epoch	Accuracy	Precision	Recall
Model A	560,612	2.1431	77.48 s	94.06%	93.91%	94.94%
Model B	9,741	0.0466	69.69 s	62.64%	51.69%	47.87%
MobileNetV2	2,351,972	9.2100	101.95 s	97.14%	97.04%	97.56%
EfficientNet-B0	4,135,648	16.0674	118.78 s	97.82%	97.60%	98.10%

4.5 Point-by-Point Comparative Analysis

4.5.1 Accuracy

EfficientNet-B0 achieved the highest test accuracy:

97.82%

MobileNetV2 achieved:

97.14%

Model A achieved:

94.06%

Model B achieved:

62.64%

Therefore, according to accuracy:

EfficientNet-B0 > MobileNetV2 > Model A > Model B

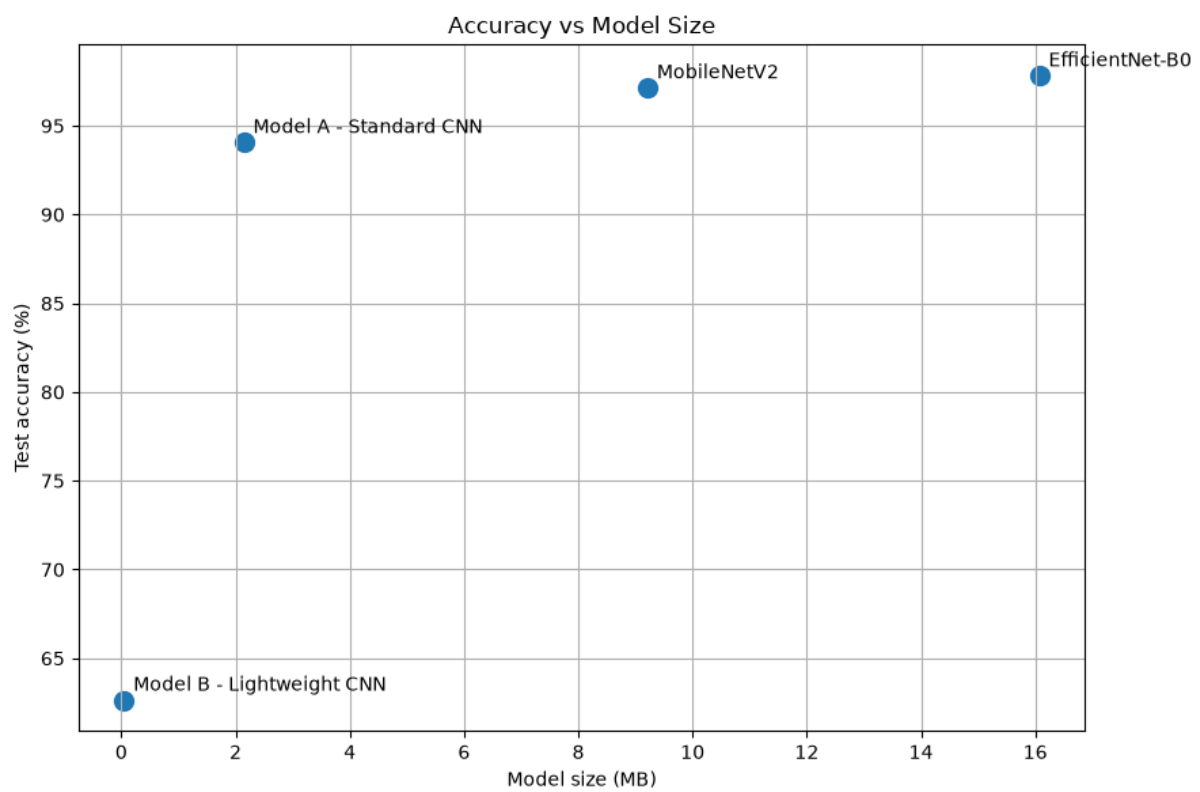


Figure 4.5: Accuracy vs Model

4.5.2 Memory Footprint

Model B has the smallest model size:

0.0466 MB

This is significantly smaller than all other evaluated models.

EfficientNet-B0 has the largest model size:

16.0674 MB

4.5.3 Computational Cost

Model B has the shortest average training time per epoch among the four models:

69.69 seconds

Model A takes:

77.48 seconds

Despite having substantially fewer parameters, Model B is faster per epoch than Model A in this experiment.

This demonstrates that parameter count alone does not determine actual execution time. Implementation efficiency, memory access, hardware utilization, and convolution implementation can all affect runtime.

4.6 Key Findings and Interpretations

The main findings are:

1. EfficientNet-B0 achieved the highest accuracy.
2. Model B achieved the smallest memory footprint.
3. Model A provides a strong middle ground.
4. MobileNetV2 provides high accuracy with a smaller footprint than EfficientNet-B0.
5. Parameter reduction can result in substantial accuracy degradation.
6. Runtime depends on both parameter count and architectural implementation.

4.7 Strategic Implications

The choice of model should depend on the requirements of the deployment environment.

For an application where maximum classification accuracy is the main objective, EfficientNet-B0 is the strongest model evaluated.

For extremely memory-constrained edge devices, Model B may be preferred despite its lower accuracy.

Model A provides a useful compromise between model size, training speed, and classification performance.

4.8 Conclusion

The experiments demonstrate that model architecture selection requires balancing accuracy, memory footprint, and computational cost.

The highest-performing model is not necessarily the most appropriate model for every deployment environment.

EfficientNet-B0 achieved the highest accuracy, while Model B achieved the lowest memory footprint.

Therefore, model selection should be based on the actual constraints and requirements of the target application.

4.9 Trade-offs, Advantages, and Limitations: Custom Model vs. Pre-trained Model

4.9.1 Custom CNN Model Analysis

Advantages

- Full control over the architecture.
- Can be specifically designed for the target problem.
- Model A has a relatively small footprint compared with the pre-trained networks.
- No dependency on ImageNet feature representations.
- Architecture can be optimized specifically for edge deployment.

Limitations

- Requires manual architecture design.
- Hyperparameter tuning may be necessary.
- Extremely small models can lose significant accuracy.
- Generalization to other datasets may be limited.

4.9.2 Pre-trained Model Analysis

Advantages

- Strong feature representations.
- High classification accuracy.
- Established architectures.
- Useful for rapid experimentation.
- Efficient transfer learning when sufficient resources are available.

Limitations

- Larger parameter count.
- Larger memory footprint.
- Higher computational cost.
- Additional input adaptation may be required.
- Pre-trained features may not be perfectly aligned with handwritten digit data.

4.9.3 Decision Framework: When to Choose Which Approach

Table 4.4: Model Selection Guidelines

Scenario	Recommended Model	Reason
Maximum classification accuracy	EfficientNet-B0	Highest test accuracy among evaluated models.
High accuracy with moderate resources	Model A	Good balance between accuracy, size, and training time.
Extremely limited memory	Model B	Only 9,741 trainable parameters and 0.0466 MB model size.
High accuracy with transfer learning	MobileNetV2	Strong accuracy with smaller footprint than EfficientNet-B0.
Strict sub-100k parameter requirement	Model B	Successfully satisfies the assignment constraint.

4.9.4 Strategic Trade-offs Summary

Accuracy vs. Memory

Higher accuracy was generally associated with larger models in this experiment.

Model B demonstrates the extreme memory-efficient end of the design space, while EfficientNet-B0 provides the highest accuracy.

Efficiency vs. Accuracy

Model A provides a practical compromise.

It achieved 94.06% accuracy with only 2.1431 MB of model storage.

Parameter Count vs. Runtime

Model B contains dramatically fewer parameters than Model A, and its average training time is also lower.

Therefore, parameter count should not be used as the only measure of computational efficiency.

4.9.5 Conclusion: Balanced Strategy

The experiments demonstrate that there is no universally optimal architecture.

EfficientNet-B0 is the best choice when classification accuracy is the highest priority and sufficient computational resources are available.

Model B is the most suitable option when memory is the dominant constraint.

Model A provides a practical middle ground between the two extremes.

Therefore, the final model should be selected based on the target deployment requirements rather than accuracy alone.

5 Overall Conclusion

This project investigated resource-constrained convolutional neural networks for classifying two-digit handwritten numbers.

Four architectures were evaluated:

1. Standard custom CNN (Model A).
2. Lightweight depthwise-separable CNN (Model B).
3. MobileNetV2.
4. EfficientNet-B0.

Model A achieved 94.06% test accuracy using 560,612 trainable parameters.

Model B successfully satisfied the assignment requirement of fewer than 100,000 trainable parameters, containing only 9,741 parameters. However, its test accuracy was 62.64%.

MobileNetV2 achieved 97.14% test accuracy.

EfficientNet-B0 achieved the highest accuracy at 97.82%.

The results demonstrate an important trade-off between classification performance and resource requirements.

For maximum accuracy, EfficientNet-B0 is the strongest choice.

For extremely constrained memory environments, Model B provides the smallest model footprint.

Model A provides a balanced alternative with strong accuracy and relatively low resource requirements.

Overall, the experiment demonstrates that designing neural networks for edge devices requires considering accuracy, memory footprint, and computational cost simultaneously.

Bibliography

- [1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [2] A. G. Howard et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” arXiv:1704.04861, 2017.
- [3] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018.
- [4] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” *Proceedings of the International Conference on Machine Learning*, 2019.
- [5] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” arXiv:1412.6980, 2014.
- [6] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016.
- [7] PyTorch Documentation, “PyTorch: An Open Source Machine Learning Framework,” Available at: <https://pytorch.org/>
- [8] Torchvision Documentation, “Torchvision Models and Datasets,” Available at: <https://pytorch.org/vision/>
- [9] PyTorch Tutorials, “Transfer Learning for Computer Vision Tutorial,” Available at: https://pytorch.org/tutorials/beginner/transfer_learning_tutorial.html

A Appendix

A.1 GitHub URL

The source code, notebooks, experiment results, and supporting files are available in the following GitHub repository:

<https://github.com/sachi1645/EN3150-Two-Digit-MNIST-Classification>

A.2 Final Model Comparison

Table A.1: Final Summary of All Evaluated Models

Model	Parameters	Size	Time/Epoch	Accuracy
Model A	560,612	2.1431 MB	77.48 s	94.06%
Model B	9,741	0.0466 MB	69.69 s	62.64%
MobileNetV2	2,351,972	9.2100 MB	101.95 s	97.14%
EfficientNet-B0	4,135,648	16.0674 MB	118.78 s	97.82%